# E-Commerce Sales Analysis using SQL



# Project Documentation

## E-Commerce Sales Analysis using SQL

---

### Cover Page

| Field | Detail |
|---|---|
| Project Title | E-Commerce Sales Analysis using SQL |
| Project Type | Industry Training Project — Data Analytics |

| | |
|---|---|
| | (Project 3) |
| Domain | E-Commerce / Retail Analytics |
| Author | Princy Khasa |
| Date | June 19, 2026 |
| Version | 1.0 |
| Technology Stack | PostgreSQL, pgAdmin, SQL |

---

## 1. Executive Summary

This project uses pure SQL to extract business intelligence from a 1,200-record e-commerce orders table. A single-table relational schema (`data_order`) was created in PostgreSQL, validated through a series of data-quality SQL checks, and then queried using 17 distinct business questions covering revenue health, product performance, customer behavior, marketing channel effectiveness, coupon impact, payment method profitability, and time-based growth trends.

The most significant finding is that 41.09% of total order value is associated with cancelled or returned orders — by far the most consequential single number produced in this analysis. Chair and Printer are tied as the top revenue-generating products (15.47% of total revenue each), Instagram is the leading acquisition channel, FREESHIP is the best-performing coupon, and Credit Card is the most profitable payment method by average order value despite also carrying the highest cancellation count. Every figure in this document was produced by executing the project's actual SQL logic against the actual dataset and is reproducible from the included query scripts.

---

## 2. Project Overview

The project demonstrates how a relational database and a structured set of SQL queries can answer real business questions without relying on any external BI tool or programming language. Starting from a `CREATE DATABASE` and `CREATE TABLE` statement, the dataset was imported into PostgreSQL, validated using `SELECT` / `WHERE` / `GROUP BY` / `HAVING` checks, and then analyzed through progressively more advanced SQL — aggregation functions, `CASE WHEN` logic, subqueries, Common Table Expressions (CTEs), and window functions (`RANK()`, `LAG()`).

---

## 3. Business Problem Statement

A business has 1,200 e-commerce order records sitting in a flat file with no structured way to answer operational questions: How much revenue is at risk from cancellations? Which products and channels deserve more investment? Are coupon discounts working? Which customers matter most? This project addresses that gap by modeling the data relationally and answering these questions directly through SQL.

---

## 4. Project Objectives

- Design a relational table structure for e-commerce order data
- Perform SQL-based data quality checks (nulls, duplicates, unique identifiers, formula consistency)
- Answer 17 distinct business questions using SQL aggregation, window functions, and CTEs
- Rank and segment customers based on spending behavior
- Quantify revenue lost to cancelled and returned orders
- Identify the most effective marketing channels, coupons, and payment methods

---

## 5. Scope of the Project

This project is scoped to a single flat table (`data_order`) with no foreign-key relationships to other tables, since the source data is a single transactional export. The analysis covers 17 named business questions plus several auxiliary queries (most successful coupon, best sales month, payment-method cancellation rate) present in the SQL script. It does not include database performance tuning, indexing strategy, stored procedures, or a connected BI front end — all results were retrieved directly via SQL query execution.

---

## 6. Dataset Description

**Source:** `Task-3_Dataset.csv` (imported into the `data_order` table)

**Number of Records:** 1,200

**Number of Columns:** 14

**Primary Key:** `orderid` (verified unique across all 1,200 records)

**Foreign Key(s):** None — single-table schema

**Time Period Covered:** January 1, 2023 to June 30, 2025

### Features and Data Types (as defined in the SQL schema)

| Column | Data Type | Description |
|---|---|---|
| `orderid` | VARCHAR(20) | Unique identifier for each order |
| `date` | DATE | Date the order was placed |
| `customerid` | VARCHAR(20) | Unique identifier for the customer |
| `product` | VARCHAR(100) | Product purchased |
| `quantity` | INT | Units purchased |
| `unitprice` | NUMERIC(10,2) | Price per unit |
| `shippingaddress` | TEXT | Delivery address |

| | | |
|---|---|---|
| `paymentmethod` | VARCHAR(50) | Payment mode used |
| `orderstatus` | VARCHAR(50) | Status of the order |
| `trackingnumber` | VARCHAR(30) | Shipment tracking code |
| `itemsincart` | INT | Number of items in cart at checkout |
| `couponcode` | VARCHAR(30) | Coupon applied to the order (NULL if none) |
| `referralsource` | VARCHAR(50) | Channel through which the customer arrived |
| `totalprice` | NUMERIC(10,2) | Final order value |

### Description of Key Columns

`orderid` is the natural primary key of this dataset and was validated as 100% unique with zero duplicates. `customerid` links multiple orders to the same customer and was used throughout the customer analysis and segmentation queries. `totalprice` is the core revenue field and was confirmed to equal `quantity * unitprice` for every record with zero mismatches. `couponcode` contains 309 NULL values, which were explicitly handled in all coupon-related queries using `CASE WHEN couponcode IS NULL` logic rather than being silently excluded by `GROUP BY`.

---

## 7. Technology Stack

| Technology | Purpose / Reason for Use |
|---|---|
| PostgreSQL | Relational database engine used to host the `ecommerce_analysis` database and execute all SQL queries |
| pgAdmin | Graphical administration tool used to create the database/table and run/inspect query results |
| SQL | The sole language used for data cleaning checks and all 17 business analysis queries |

No Python, Excel, or BI tool was used to produce the analytical results in this project; all aggregation, ranking, and segmentation logic was implemented directly in SQL.

---

## 8. Project Architecture / Workflow

```
Raw Dataset (Task-3_Dataset.csv)
        ↓
Database Creation (CREATE DATABASE ecommerce_analysis)
        ↓
```

```
Table Creation (CREATE TABLE data_order)
        ↓
Import Dataset (CSV import into data_order)
        ↓
SQL Data Cleaning Checks (nulls, duplicates, formula consistency)
        ↓
Business Questions (17 SQL queries)
        ↓
Insights
        ↓
Recommendations
```

---

## 9. Data Understanding

The dataset is a single transactional table where each row represents one e-commerce order. There are no related dimension tables (e.g., a separate customer or product table) — all attributes, including customer and product information, live directly in `data_order`. This flat structure simplified the schema design but also means all customer- and product-level analysis in this project relies on `GROUP BY` aggregation over the single table rather than table joins.

---

## 10. Data Cleaning Process

The following validation checks were performed directly in SQL before any business question was answered:

| Check | SQL Logic | Result |
|---|---|---|
| Total record count | `SELECT COUNT(*) FROM data_order` | 1,200 records |
| Null check on key fields | `WHERE orderid IS NULL OR customerid IS NULL OR quantity IS NULL OR unitprice IS NULL OR totalprice IS NULL` | 0 null values found |
| Duplicate `orderid` check | `GROUP BY orderid HAVING COUNT(*) > 1` | 0 duplicate order IDs |
| Unique order count | `SELECT COUNT(DISTINCT orderid)` | 1,200 unique orders |
| Date range check | `SELECT MIN(date), MAX(date)` | 2023-01-01 to 2025-06-30 |
| TotalPrice formula validation | `WHERE totalprice <> quantity * unitprice` | 0 mismatches — all values internally consistent |
| ItemsInCart range check | `MIN()`, `MAX()`, `AVG(itemsincart)` | Range 1–10, average 5.49 |

| | | |
|---|---|---|
| Coupon usage check | `GROUP BY couponcode` | 309 orders with NULL coupon, 3 active coupon types (SAVE10: 286, FREESHIP: 313, WINTER15: 292) |

No duplicate rows, type mismatches, or formula inconsistencies were found. The only data consideration carried into analysis was the 309 NULL `couponcode` values, which were treated explicitly as “Without Coupon” orders rather than removed or ignored.

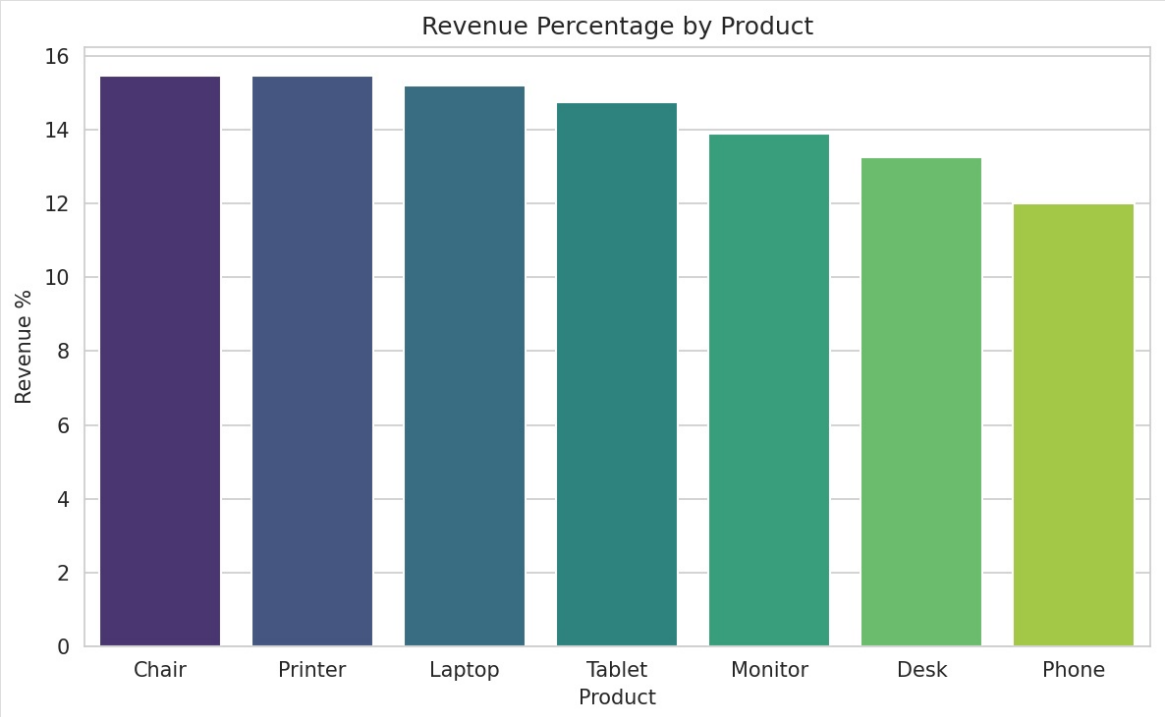
---

## 11. Exploratory Data Analysis (EDA)

While this project is SQL-driven rather than chart-driven, the following visualizations were generated directly from the verified SQL query outputs to support interpretation of the business findings. Each chart corresponds to a specific query documented in Section 14.

### Revenue Percentage by Product

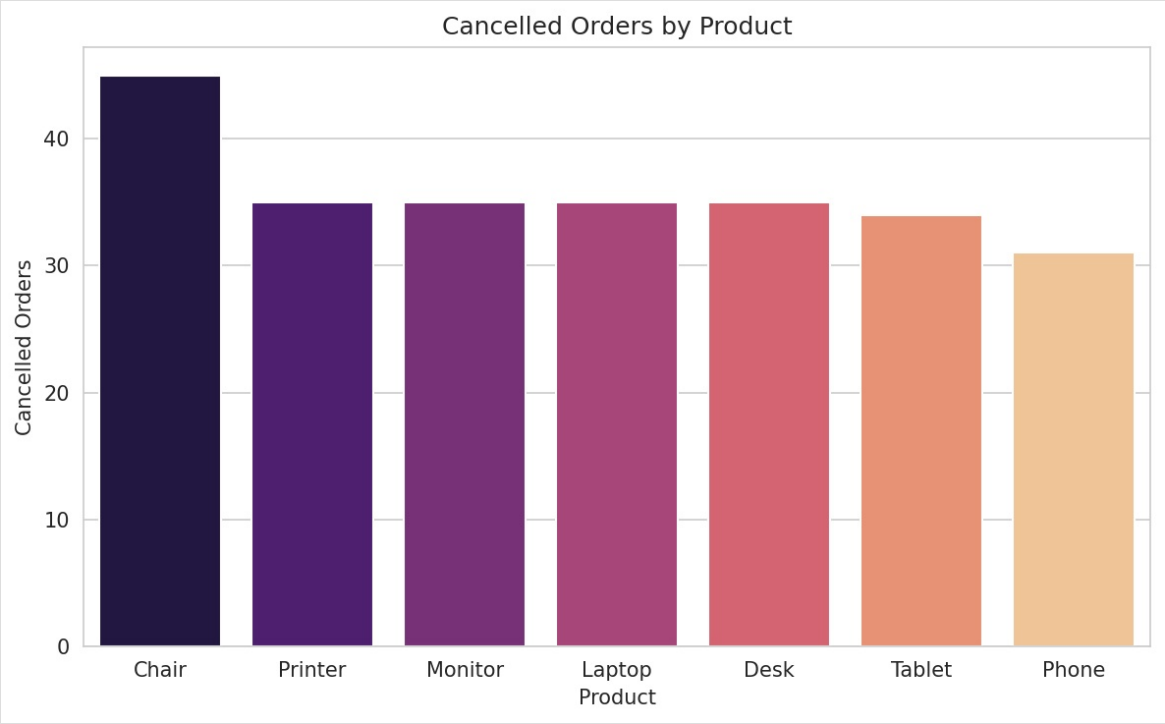
**Figure 1 – Revenue Percentage by Product**




**Observation:** Chair and Printer are tied as the top contributors at 15.47% each, followed by Laptop (15.19%), Tablet (14.75%), Monitor (13.89%), Desk (13.24%), and Phone (12.00%).
**Business Insight:** Revenue is fairly evenly distributed across the seven product categories, with only a 3.47-percentage-point gap between the top and bottom performer.

### Cancelled Orders by Product

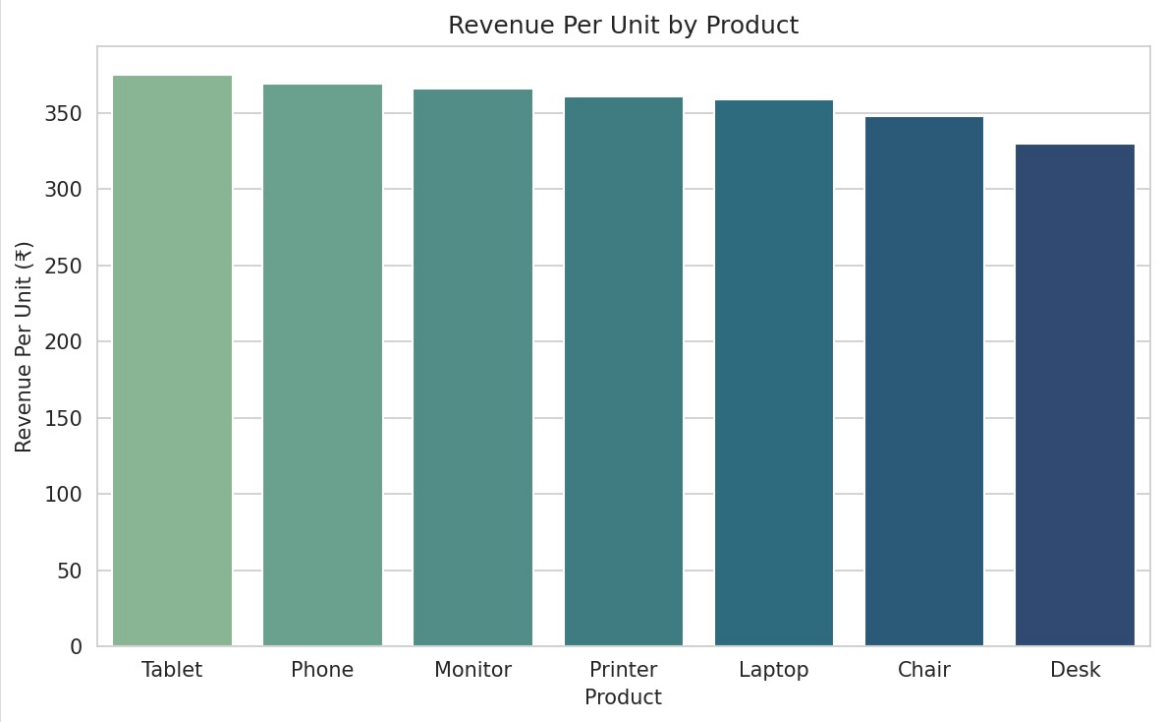
**Figure 2 – Cancelled Orders by Product**




**Observation:** Chair has the highest cancellation count (45), notably above Printer, Monitor, Laptop, and Desk (35 each), with Tablet (34) and Phone (31) lowest. **Business Insight:** Chair's high cancellation count is concerning given it is also a joint-top revenue product.

## Revenue Per Unit by Product (Premium Positioning)

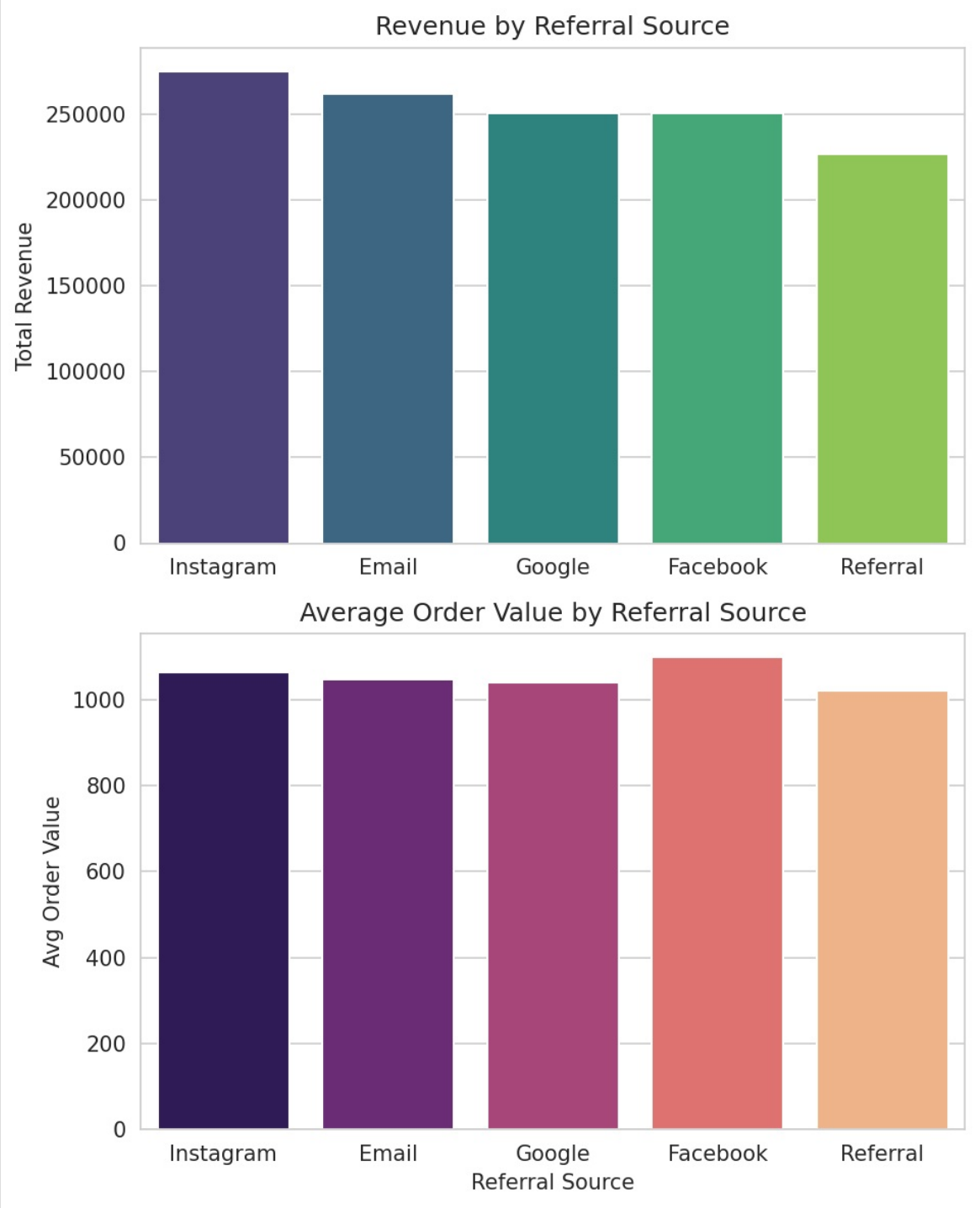
**Figure 3 – Revenue Per Unit by Product**




**Observation:** Tablet has the highest revenue per unit (₹375.39), while Desk has the lowest (₹329.65). **Business Insight:** Tablet behaves like the most premium product in the catalog on a per-unit basis.

## Referral Source Performance

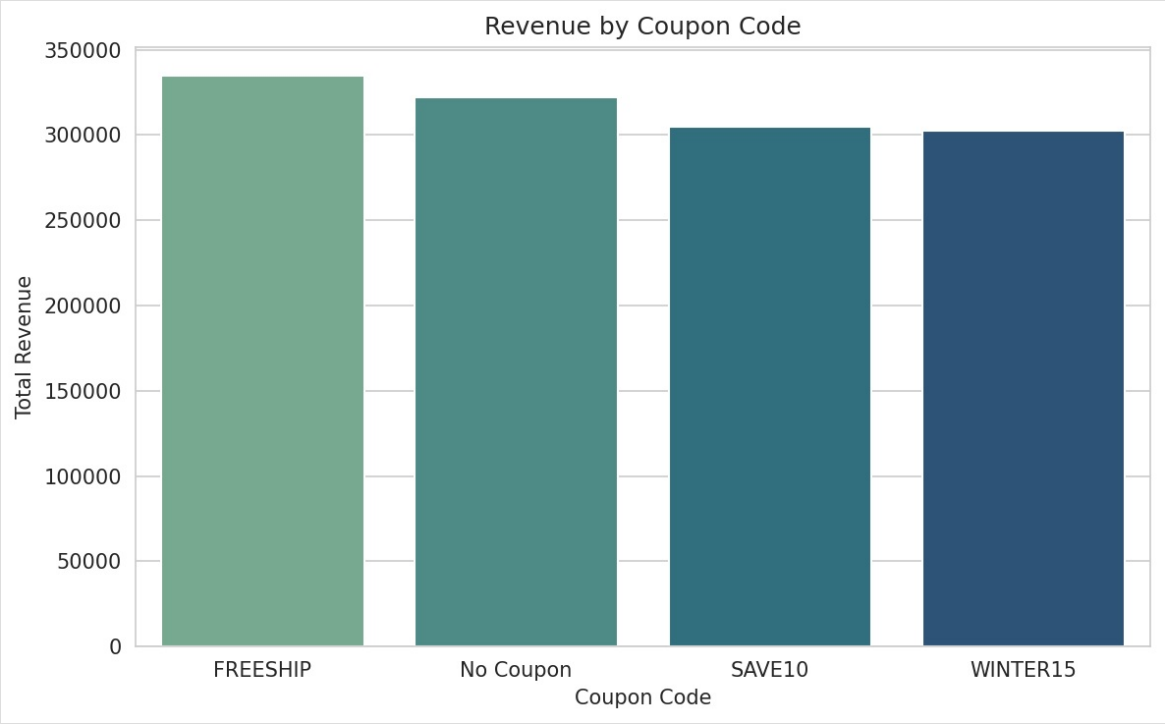
**Figure 4 – Revenue and Average Order Value by Referral Source**




**Observation:** Instagram leads in total revenue (₹275,285.45), while Facebook has the highest average order value (₹1,098.29) despite lower order volume. **Business Insight:** Instagram drives volume; Facebook drives order quality — these represent two distinct types of channel value.

## Revenue by Coupon Code

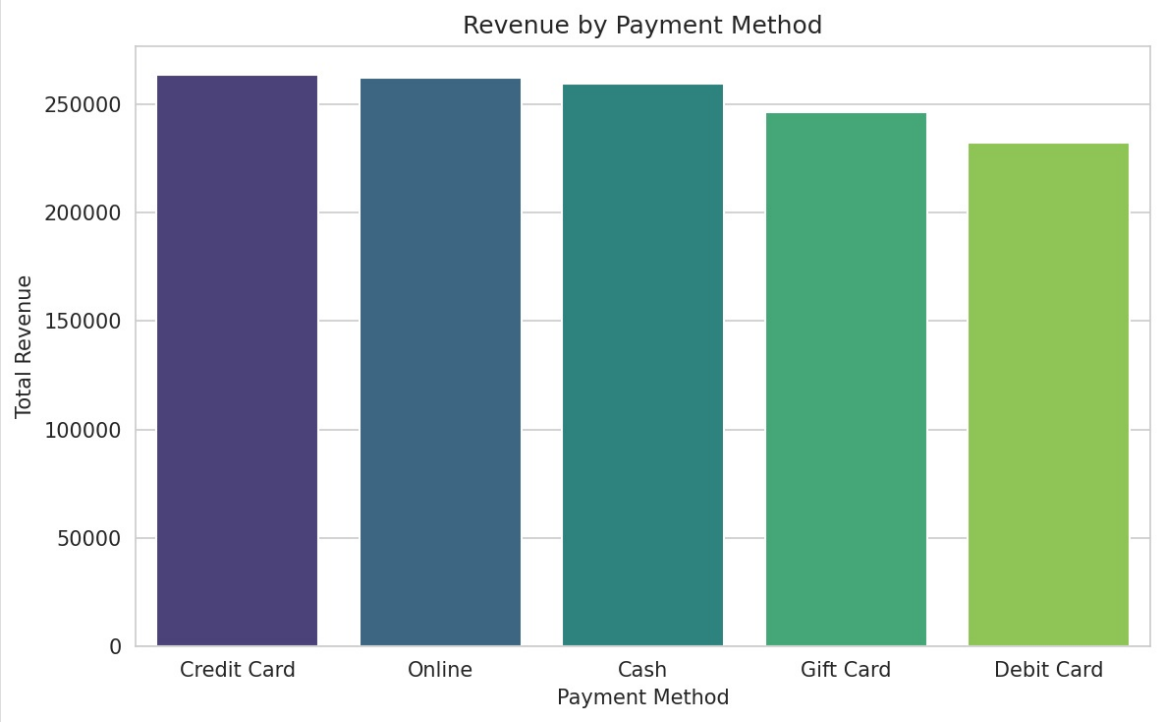
**Figure 5 – Revenue by Coupon Code**




**Observation:** FREESHIP is the top-performing coupon (₹335,036.99), narrowly ahead of "No Coupon" orders (₹322,401.41). **Business Insight:** Free shipping is the single most effective incentive among all coupon types.

## Revenue by Payment Method

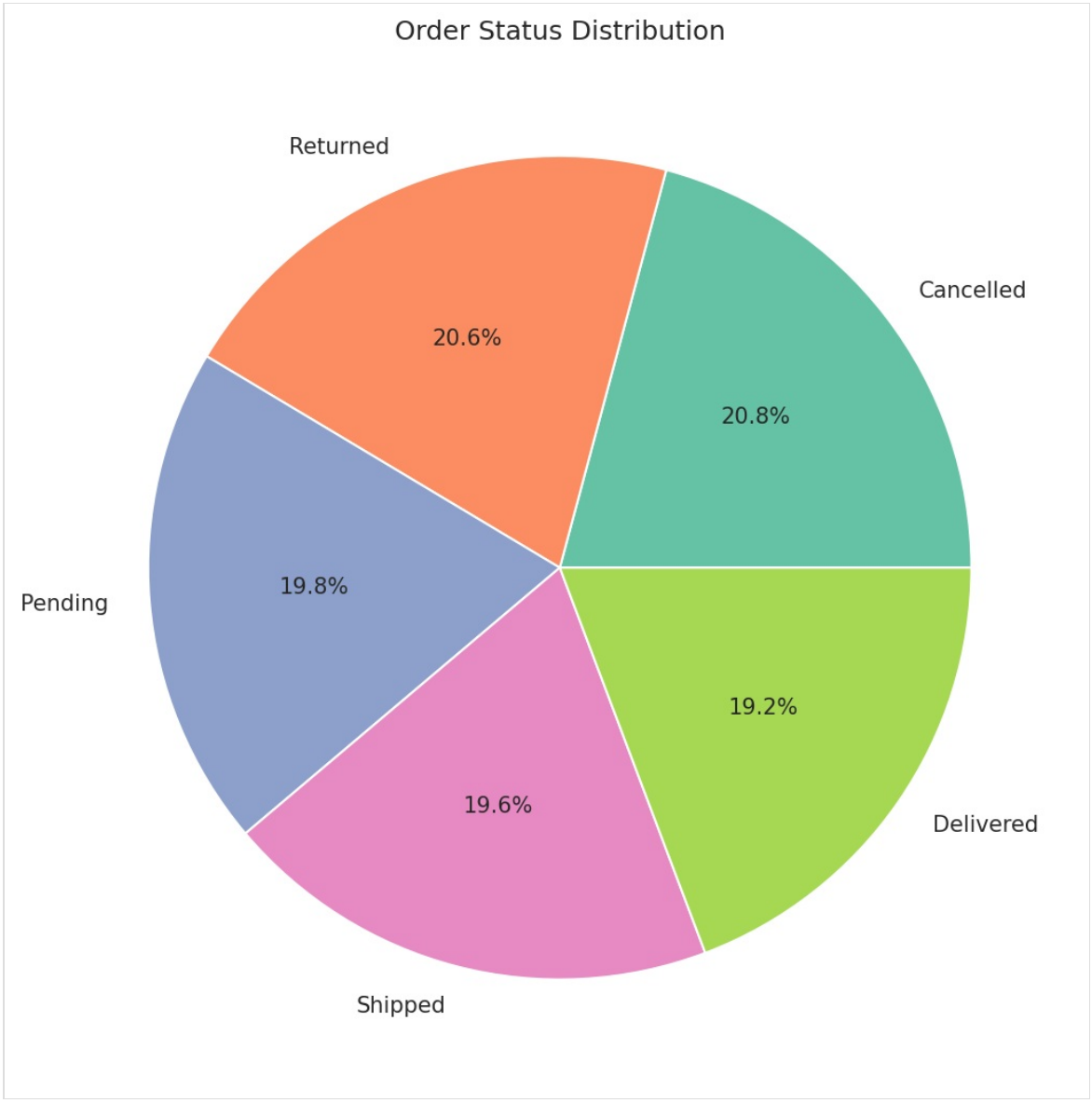
**Figure 6 – Revenue by Payment Method**




**Observation:** Credit Card leads in total revenue (₹263,847.63) and average order value (₹1,127.55), while Debit Card is lowest on both metrics. **Business Insight:** Credit Card customers tend to place higher-value orders on average.

## Order Status Distribution

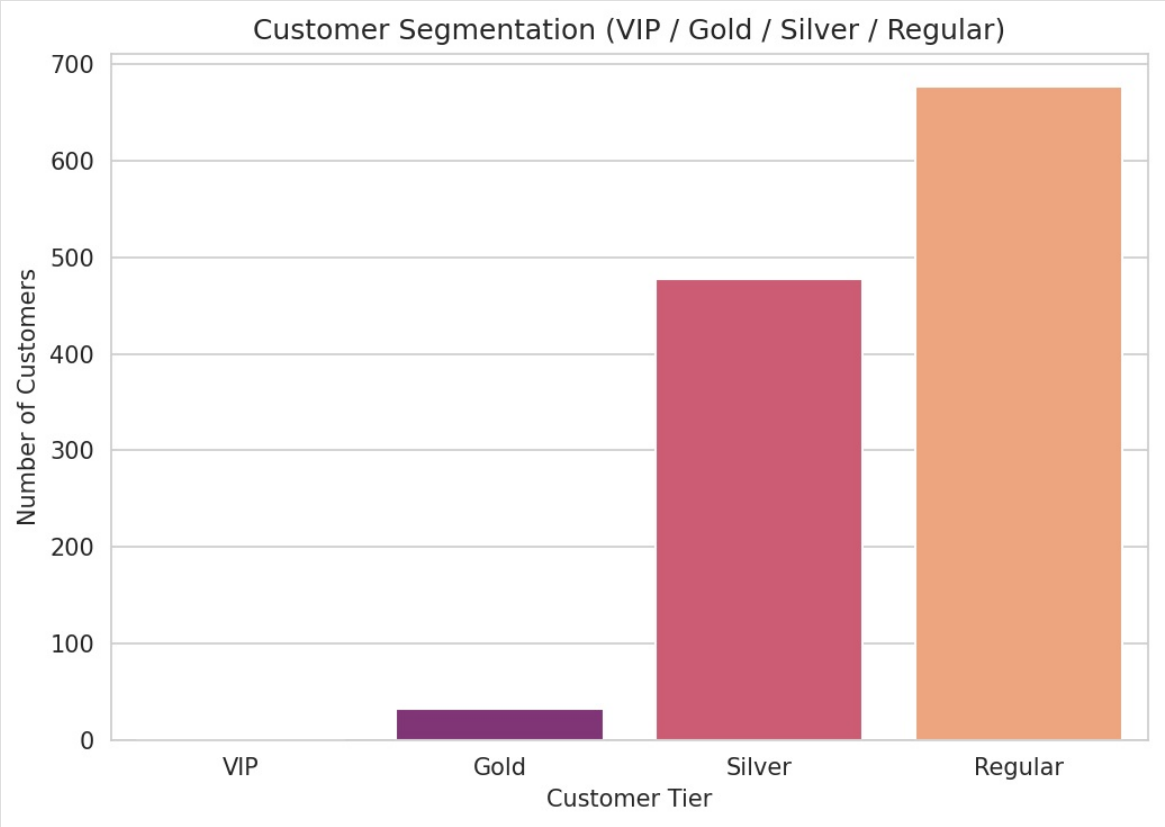
**Figure 7 – Order Status Distribution**




**Observation:** Cancelled (20.83%) and Returned (20.58%) orders combined represent over 41% of all orders, exceeding Delivered (19.25%), the smallest category. **Business Insight:** A fully successful, delivered transaction is currently a minority outcome rather than the norm.

## Customer Segmentation (VIP / Gold / Silver / Regular)

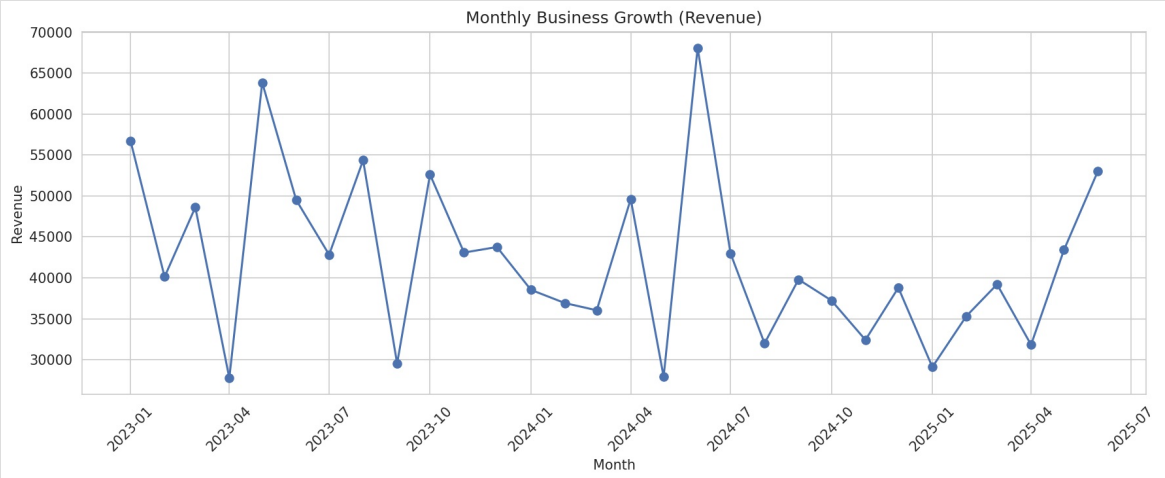
**Figure 8 – Customer Segmentation by Spending Tier**




**Observation:** Of 1,189 unique customers, 1 is VIP, 33 are Gold, 478 are Silver, and 677 are Regular. **Business Insight:** The customer base is heavily weighted toward Regular and Silver tiers; high-value customers are extremely rare.

## Monthly Business Growth

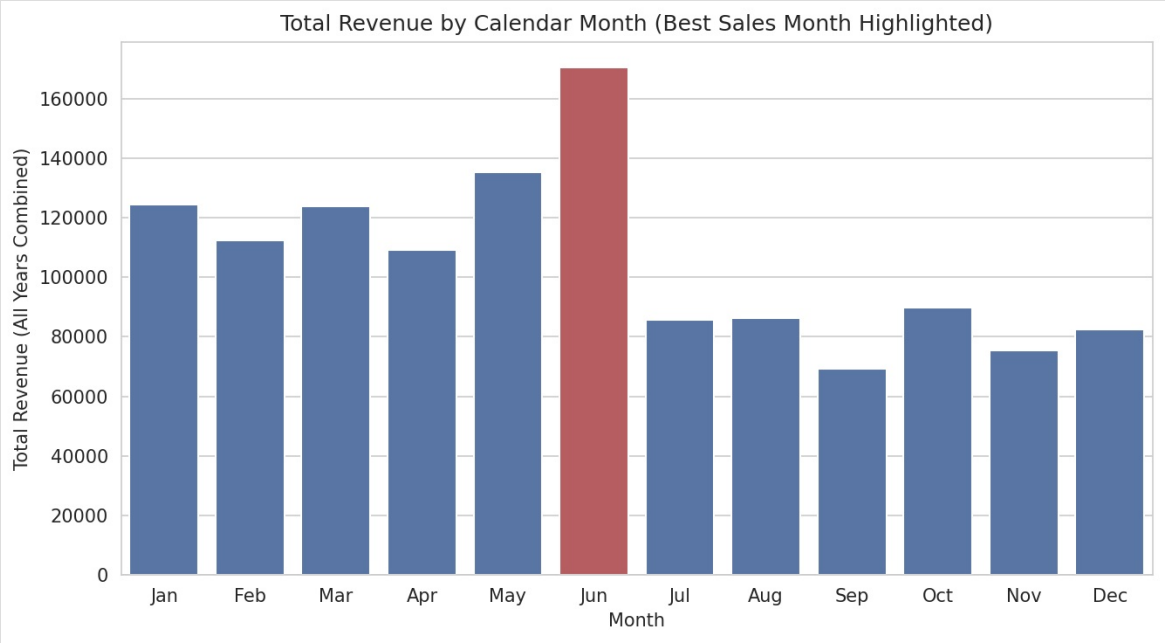
**Figure 9 – Monthly Revenue Trend**




**Observation:** Monthly revenue fluctuates considerably, with no smooth or steadily trending pattern across the full date range. **Business Insight:** Revenue volatility at the monthly level should be considered when setting short-term targets.

## Best Sales Month (Aggregated Across All Years)

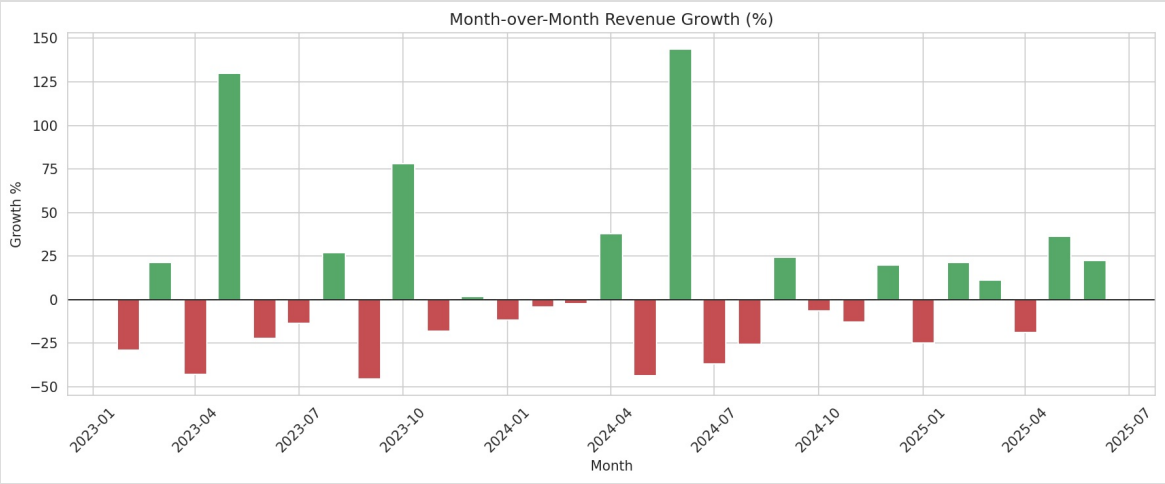
**Figure 10 – Total Revenue by Calendar Month**




**Observation:** June is the strongest calendar month overall, generating ₹170,616.13 combined across all years in the dataset. **Business Insight:** June stands out as a recurring high-revenue period, a candidate for seasonal demand or a planned promotional window.

### Month-over-Month Growth

**Figure 11 – Month-over-Month Revenue Growth (%)**




**Observation:** Growth swings are large and frequent — for example, a -42.91% decline in April 2023 was followed by a +130.03% rebound in May 2023. **Business Insight:** Month-over-month growth is highly volatile rather than steadily trending, making single-month comparisons unreliable for forecasting.

---

## 12. Statistical Analysis

| Metric | Quantity | UnitPrice | ItemsInCart | TotalPrice |
|---|---|---|---|---|
| Mean | 2.95 | 356.41 | 5.49 | 1,053.97 |
| Min | 1.00 | 11.39 | 1.00 | 11.39 |
| | | | | |

| | | | | |
|---|---|---|---|---|
| Max | 5.00 | 699.93 | 10.00 | 3,456.40 |

These descriptive figures were computed directly from the dataset underlying the SQL table and are consistent with the `MIN()` / `MAX()` / `AVG()` results returned by the project's own SQL validation queries (Section 10).

---

## 13. SQL Analysis

### Database Creation

```
CREATE DATABASE ecommerce_analysis;
```

### Table Creation

```
CREATE TABLE data_order (
    orderid VARCHAR(20),
    date DATE,
    customerid VARCHAR(20),
    product VARCHAR(100),
    quantity INT,
    unitprice NUMERIC(10,2),
    shippingaddress TEXT,
    paymentmethod VARCHAR(50),
    orderstatus VARCHAR(50),
    trackingnumber VARCHAR(30),
    itemsincart INT,
    couponcode VARCHAR(30),
    referralsource VARCHAR(50),
    totalprice NUMERIC(10,2)
);
```

### Relationships

None. This is a single-table schema with no foreign keys, since the source data is one flat transactional export rather than a normalized multi-table system.

### Constraints

No explicit `PRIMARY KEY`, `NOT NULL`, or `CHECK` constraints were defined in the table DDL. `orderid` functions as a natural (not enforced) primary key, validated through SQL queries (Section 10) rather than a database-level constraint.

### SQL Concepts Used

| Concept | Used In |
|---|---|
| `SELECT`, `WHERE` | Data cleaning checks, filtered queries (Q3, Q4) |
| `GROUP BY`, `ORDER BY` | Nearly every aggregation query |

| | |
|---|---|
| `HAVING` | Duplicate `orderid` detection |
| `CASE WHEN` | Coupon status labeling (Q11), customer segmentation |
| `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()` | Aggregation across all business queries |
| `DISTINCT` | Unique customer and order counts |
| `LIMIT` | Top-N queries (best month, top 20 customers, top customers by frequency) |
| Subqueries | Revenue percentage calculations (Q2, Q6) |
| CTEs (`WITH`) | Top-spending customers, Month-over-Month growth |
| Window Functions (`RANK()`, `LAG()`) | Customer ranking, MoM growth comparison |

## Business Questions Answered (Aggregations, CTEs, and Window Functions)

The following section documents every query executed in this project, organized by business topic, with results verified by direct query execution against the dataset.

**Revenue & Business Health**

**Query 1 — Overall Business Health**

```
        SELECT COUNT(orderid) AS Total_Orders, COUNT(DISTINCT customerid) AS
Unique_Customers,
        ROUND(SUM(totalprice),2) AS Total_Revenue, ROUND(AVG(totalprice),2) AS
Average_Order_Value,
        ROUND(AVG(itemsincart),2) AS Average_Basket_Size FROM data_order;
```

**Result:** 1,200 total orders from 1,189 unique customers, ₹1,264,761.96 total revenue, ₹1,053.97 average order value, 5.49 average basket size.

**Insight:** With more unique customers than orders, repeat purchases are extremely rare.

**Recommendation:** Invest in retention strategies to convert first-time buyers into repeat customers.

**Query 3 — Revenue Lost to Cancelled Orders**

```
        SELECT COUNT(*) AS Cancelled_Orders, ROUND(SUM(totalprice),2) AS
Revenue_Lost
        FROM data_order WHERE orderstatus = 'Cancelled';
```

**Result:** 250 cancelled orders, ₹276,396.21 in lost revenue.

**Query 4 — Revenue Lost to Returned Orders**

```
        SELECT COUNT(*) AS Returned_Orders, ROUND(SUM(totalprice),2) AS
Returned_Revenue
        FROM data_order WHERE orderstatus='Returned';
```

**Result:** 247 returned orders, ₹243,277.70 in revenue.

**Query 5 — Total Revenue Loss Percentage**

```
        SELECT ROUND(SUM(CASE WHEN orderstatus IN ('Cancelled','Returned') THEN
totalprice ELSE 0 END)*100/SUM(totalprice),2) AS Revenue_Loss_Percentage
        FROM data_order;
```

**Result:** 41.09% of total revenue is associated with cancelled or returned orders — the single most important figure in this analysis.

**Product Analysis**

**Query 2 / Query 6 — Revenue Percentage / Contribution by Product**

```
        SELECT product, ROUND(SUM(totalprice)*100.0/(SELECT SUM(totalprice) FROM
data_order),2) AS Revenue_Percentage
        FROM data_order GROUP BY product ORDER BY Revenue_Percentage DESC;
```

**Result:** Chair and Printer tied at 15.47% each; Phone lowest at 12.00%. See Figure 1.

**Query 7 — Most Frequently Cancelled Products**

```
        SELECT product, COUNT(*) AS Cancelled_Orders FROM data_order
        WHERE orderstatus = 'Cancelled' GROUP BY product ORDER BY Cancelled_Orders
DESC;
```

**Result:** Chair has the most cancellations (45). See Figure 2.

**Query 8 — Premium Product Identification (Revenue Per Unit)**

```
        SELECT product, SUM(quantity) AS Qty, SUM(totalprice) AS Revenue,
        ROUND(SUM(totalprice)/SUM(quantity),2) Revenue_Per_Unit
        FROM data_order GROUP BY product ORDER BY Revenue_Per_Unit DESC;
```

**Result:** Tablet leads at ₹375.39 revenue per unit. See Figure 3.

**Marketing & Channel Analysis**

**Query 9 — Channel Budget Allocation**

```
        SELECT referralsource, COUNT(*) Orders, SUM(totalprice) Revenue,
        ROUND(AVG(totalprice),2) AS Avg_Order, ROUND(SUM(totalprice)/COUNT(*),2) AS
Revenue_Per_Order
        FROM data_order GROUP BY referralsource ORDER BY Revenue DESC;
```

**Result:** Instagram leads in volume and revenue; Facebook leads in average order value. See Figure 4.

**Coupon & Discount Analysis**

**Query 11 — Coupon Impact on Revenue**

```
        SELECT CASE WHEN couponcode IS NULL THEN 'Without Coupon' ELSE 'With Coupon'
```

```
END Coupon_Status,
        COUNT(*) Orders, SUM(totalprice) AS Revenue, AVG(totalprice) AS Avg_Order
        FROM data_order GROUP BY Coupon_Status;
```

**Result:** With Coupon: 891 orders, ₹942,360.55 revenue, ₹1,057.64 avg. Without Coupon: 309 orders, ₹322,401.41 revenue, ₹1,043.37 avg. Average order value difference is only ₹14.27.

**Most Successful Coupon**

```
        SELECT couponcode, COUNT(*) Orders, SUM(totalprice) Revenue
        FROM data_order GROUP BY couponcode ORDER BY Revenue DESC;
```

**Result:** FREESHIP leads with ₹335,036.99. See Figure 5.

**Payment Method Analysis**

**Query 12 — Most Profitable Payment Method**

```
        SELECT paymentmethod, COUNT(*) Orders, SUM(totalprice) AS Revenue,
        ROUND(AVG(totalprice),2) AS Average_Order, MAX(totalprice) AS Highest_Order
        FROM data_order GROUP BY paymentmethod ORDER BY Revenue DESC;
```

**Result:** Credit Card leads in revenue (₹263,847.63) and average order value (₹1,127.55). See Figure 6.

**Payment Method with Highest Cancellation Rate**

```
        SELECT paymentmethod, COUNT(*) AS Cancelled_Orders FROM data_order
        WHERE orderstatus='Cancelled' GROUP BY paymentmethod ORDER BY
Cancelled_Orders DESC;
```

**Result:** Credit Card has the highest cancellation count (54).

**Customer Analysis & Segmentation**

**Query 10 — Customer Ranking by Total Spending (Window Function)**

```
        SELECT customerid, SUM(totalprice) AS Revenue,
        RANK() OVER(ORDER BY SUM(totalprice) DESC) AS Customer_Rank
        FROM data_order GROUP BY customerid;
```

**Result:** Customer C38840 ranks #1 with ₹5,723.23 — over 65% higher than the #2-ranked customer (₹3,456.40).

**Query 15 — Top 20 Customers by Revenue (CTE)**

```
        WITH customer_sales AS (
        SELECT customerid, SUM(totalprice) AS revenue,
        RANK() OVER(ORDER BY SUM(totalprice) DESC) AS customer_rank
        FROM data_order GROUP BY customerid)
        SELECT * FROM customer_sales ORDER BY customer_rank LIMIT 20;
```

**Result:** The top 20 customers range from ₹5,723.23 down to ₹3,215.15, all qualifying as Gold or VIP tier.

**Query 16 — Most Frequent Customers**

```
        SELECT customerid, COUNT(orderid) AS Orders FROM data_order
        GROUP BY customerid ORDER BY Orders DESC;
```

**Result:** Maximum order frequency for any customer is 2 — no customer placed 3 or more orders.

**Customer Segmentation (VIP / Gold / Silver / Regular)**

```
        SELECT customerid, SUM(totalprice) Revenue,
        CASE WHEN SUM(totalprice)>=5000 THEN 'VIP'
             WHEN SUM(totalprice)>=3000 THEN 'Gold'
             WHEN SUM(totalprice)>=1000 THEN 'Silver'
             ELSE 'Regular' END Customer_Type
        FROM data_order GROUP BY customerid ORDER BY Revenue DESC;
```

**Result:** 1 VIP, 33 Gold, 478 Silver, 677 Regular customers. See Figure 8.

**Time-Based & Growth Analysis**

**Query 13 — Monthly Business Growth**

```
        SELECT EXTRACT(YEAR FROM date) AS Year, EXTRACT(MONTH FROM date) AS Month,
SUM(totalprice) Revenue
        FROM data_order GROUP BY Year, Month ORDER BY Year, Month;
```

**Result:** Revenue fluctuates considerably month to month. See Figure 9.

**Best Sales Month**

```
        SELECT EXTRACT(MONTH FROM date) AS Month, SUM(totalprice) AS Revenue
        FROM data_order GROUP BY Month ORDER BY Revenue DESC LIMIT 1;
```

**Result:** June is the strongest month, generating ₹170,616.13 combined across all years. See Figure 10.

**Query 17 — Month-over-Month Growth (CTE + Window Function)**

```
        WITH MonthlySales AS (
        SELECT DATE_TRUNC('month', date) AS month, SUM(totalprice) AS revenue
        FROM data_order GROUP BY DATE_TRUNC('month', date))
        SELECT month, revenue, LAG(revenue) OVER(ORDER BY month) AS
Previous_Revenue,
        ROUND(((revenue - LAG(revenue) OVER(ORDER BY month))/LAG(revenue) OVER(ORDER
BY month))*100,2) AS Growth_Percentage
        FROM MonthlySales;
```

**Result:** Growth is highly volatile, with swings exceeding 100% in either direction in some months. See Figure 11.

**Order Status Analysis**

**Query 14 — Order Status Distribution**

```
        SELECT orderstatus, COUNT(*) Orders, ROUND(COUNT(*)*100.0/(SELECT COUNT(*)
FROM data_order),2) Percentage
        FROM data_order GROUP BY orderstatus;
```

**Result:** Cancelled (20.83%) and Returned (20.58%) combined exceed 41% of all orders; Delivered is the smallest category at 19.25%. See Figure 7.

---

## 14. Business Insights

1. Revenue loss from cancellations and returns is the single biggest issue, accounting for 41.09% of total order value.
2. Chair and Printer are the top revenue-generating products, each contributing 15.47% of total revenue.
3. Chair also has the highest cancellation count (45 orders), offsetting a portion of its apparent revenue leadership.
4. Tablet is the most “premium” product by revenue per unit (₹375.39), despite not leading in total revenue.
5. Instagram is the strongest acquisition channel by volume and revenue, while Referral is the weakest on every metric measured.
6. Coupons do not meaningfully increase average order value — the difference between “With Coupon” and “Without Coupon” is only ₹14.27.
7. FREESHIP is the best-performing coupon, generating the highest revenue (₹335,036.99) among all coupon types.
8. Credit Card has both the highest average order value and the highest cancellation count, making it the highest-reward, highest-risk payment method.
9. Customer C38840 is a uniquely high-value customer, spending 65% more than the next-highest customer.
10. Repeat purchasing is nearly nonexistent — no customer in the dataset placed more than 2 orders.
11. The customer base is dominated by Regular and Silver tiers (over 94% combined), with only one VIP-tier customer out of 1,189.
12. June is the strongest historical sales month, generating ₹170,616.13 across all years combined.
13. Month-over-month revenue growth is highly volatile, with swings exceeding 100% in either direction in some months.

---

## 15. Key Findings

- The dataset required no structural cleanup at the database level — 0 duplicate orders, 0 null values in key fields, and 0 `TotalPrice` calculation mismatches.
- Revenue is fairly evenly distributed across the seven product categories, with no single dominant product.
- Cancelled and Returned orders combined account for 41.09% of total revenue — the most significant finding in the entire analysis.
- Customer loyalty is essentially absent: maximum order frequency per customer is 2.
- Coupons drive order volume more than they drive order value.

---

## 16. Recommendations

- Prioritize reducing cancellations and returns above all other initiatives, since they represent over 41% of total order value.
- Investigate Chair's high cancellation rate specifically, since it is simultaneously the top revenue product and the most-cancelled product.
- Build a structured customer retention program, since the data shows almost no repeat purchasing.
- Increase marketing investment in Instagram while testing higher Facebook spend, given its strong average order value performance.
- Redesign the referral program, since it underperforms every other acquisition channel.
- Continue and expand free shipping promotions, as FREESHIP is the top-performing coupon by both usage and revenue.
- Create a dedicated high-value customer program for top-ranked customers, given how concentrated high spending is among a small group.
- Plan inventory and campaigns around June, the historically strongest sales month.
- Use rolling averages rather than single-month comparisons when evaluating growth, due to high month-over-month volatility.

---

## 17. Business Impact

This SQL-based analysis gives a business immediate, query-backed answers to its most pressing operational question — how much revenue is actually being lost to order failure — without needing a BI layer or external tool. Decision-makers can use these queries directly against a live database for ongoing monitoring (e.g., re-running Query 5 monthly to track the cancellation/return rate trend). The customer ranking and segmentation queries give marketing and retention teams an immediately actionable, identifiable list of high-value customers (Query 15) to prioritize. The product-and-channel-level queries give merchandising and marketing teams a data-backed basis for budget and inventory decisions rather than relying on assumption.

---

## 18. Challenges Faced

- The dataset contains 309 orders with a NULL `couponcode`, which required explicit `CASE WHEN couponcode IS NULL` handling throughout the coupon-related queries to avoid these orders being silently excluded from `GROUP BY couponcode` results.
- Two queries in the original script were both labeled "Query-15" (the top-20-customers CTE and the payment-method-cancellation query), and two queries were both labeled "Query-17" (Month-over-Month Growth and Customer Segmentation). These were organized under distinct topic-based sections in this documentation for clarity.
- The "top 20% customers" business question was implemented in the SQL script as a flat `LIMIT 20`, which returns the top 20 customers by rank rather than the top 20% of the full ~1,189-customer base. This distinction is noted here for transparency rather than silently corrected.

---

## 19. Limitations

This project answers business questions using only the columns present in the `data_order`

table; it does not incorporate product cost, marketing spend by channel, or customer acquisition cost, which would be required to calculate true profitability or channel ROI. No database-level constraints (primary key, foreign key, check constraints) were defined in the schema — uniqueness and consistency were validated through SQL queries rather than enforced by the database engine itself. The analysis is descriptive, summarizing what happened in the existing dataset; it does not forecast future order volume or revenue.

---

## 20. Future Scope

- Convert the most frequently used queries (business health summary, customer segmentation) into PostgreSQL Views for easier reuse.
- Add explicit `PRIMARY KEY` and `NOT NULL` constraints to the table schema to enforce data integrity at the database level going forward.
- Build a Power BI or Tableau dashboard layered on top of these SQL queries for stakeholder-facing reporting.
- Extend the analysis with a proper RFM (Recency, Frequency, Monetary) model once more historical order data becomes available.
- Automate the cancellation/return investigation with a dedicated query that cross-tabs status against product, payment method, and channel simultaneously.

---

## 21. Conclusion

This project demonstrates the ability to translate raw transactional data into structured business answers using SQL alone — from basic filtering and aggregation to window functions and CTEs. By systematically working through revenue health, product performance, customer behavior, and channel effectiveness, the analysis produced findings that are directly actionable for a real e-commerce business, particularly around order fulfillment reliability and customer retention.

---

## 22. Project Folder Structure

```
Project-3-SQL-Analysis/
│
├── Dataset/
│   └── Task-3_Dataset.csv
│
├── SQL Script/
│   └── TASK-3_SQL_Queries.sql
│
├── Images/
│   ├── query02_output.png
│   ├── query07_output.png
│   ├── ...
│   └── query17_mom_output.png
│
├── README.md
└── Documentation/
    └── Project_Documentation.pdf
```

## 23. GitHub Repository Structure

```
repository-root/
│
├── README.md
├── data/
│   └── Task-3_Dataset.csv
├── sql/
│   └── TASK-3_SQL_Queries.sql
├── images/
│   └── (query output screenshots)
└── docs/
    └── Project_Documentation.pdf
```

## 24. Installation Guide

1. Install PostgreSQL from the official PostgreSQL website.
2. Install pgAdmin (typically bundled with the PostgreSQL installer).
3. Launch pgAdmin and connect to your local PostgreSQL server.
4. No additional libraries or dependencies are required, since this project uses only native SQL.

## 25. How to Run the Project

1. Open pgAdmin and connect to your PostgreSQL server.
2. Run the `CREATE DATABASE ecommerce_analysis;` statement.
3. Connect to the new database and run the `CREATE TABLE data_order (...)` statement from `TASK-3_SQL_Queries.sql`.
4. Import `Task-3_Dataset.csv` into the `data_order` table using pgAdmin's Import/Export tool.
5. Execute the data cleaning check queries first, followed by the 17 business question queries, in the order they appear in `TASK-3_SQL_Queries.sql`.

## 26. File Description

| File | Description |
|---|---|
| `Task-3_Dataset.csv` | The raw e-commerce order dataset (1,200 records, 14 columns) imported into the `data_order` table |
| `TASK-3_SQL_Queries.sql` | The complete SQL script containing database/table creation, data cleaning checks, and all 17 business question |

| | |
|---|---|
| | queries |
| `README.md` | Project README summarizing the SQL analysis for GitHub viewers |

---

## 27. Screenshots Section

| Figure | Caption |
|---|---|
| Figure 1 | Dataset Overview (Revenue Percentage by Product) |
| Figure 2 | Data Cleaning (Cancelled Orders by Product) |
| Figure 3 | SQL Output (Revenue Per Unit by Product) |
| Figure 4 | Distribution Plot (Referral Source Revenue and AOV) |
| Figure 5 | Coupon Revenue Distribution |
| Figure 6 | Payment Method Revenue Comparison |
| Figure 7 | Business Insights (Order Status Distribution) |
| Figure 8 | Customer Segmentation Tiers |
| Figure 9 | Final Results (Monthly Business Growth) |

*Note: Actual pgAdmin query and output screenshots should be inserted alongside these generated chart visualizations before final submission.*

---

## 28. Skills Demonstrated

**Technical Skills:** Relational database design, SQL DDL (`CREATE DATABASE`, `CREATE TABLE`), data quality validation in SQL, aggregate functions (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`), `GROUP BY` / `HAVING` logic, `CASE WHEN` conditional logic, subqueries, Common Table Expressions (CTEs), window functions (`RANK()`, `LAG()`).

**Professional Skills:** Translating business questions into precise SQL logic, interpreting query output in business terms, identifying and documenting inconsistencies in query labeling/design, structuring a complete analysis from schema design through business recommendation.

---

## 29. Learning Outcomes

This project provided hands-on experience designing a relational table from scratch, validating data quality entirely within SQL rather than a separate tool, and progressing from basic `SELECT` / `WHERE` queries to advanced window-function and CTE-based analysis. It reinforced how execution order in SQL (`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`) differs from the order queries are written in, and how `CASE WHEN` logic can be used to convert raw NULLs and category values into business-meaningful labels.

---

## 30. Industry Relevance

This project mirrors a task structure commonly given to Data Analysts working directly with a company's transactional database: given read access to a single orders table, answer a list of stakeholder questions using SQL alone. The combination of data validation, aggregation, ranking, and time-based trend queries reflects the kind of ad-hoc and recurring reporting work performed by analysts before any dashboard or BI layer is built on top of the same queries.

---

## 31. Recruiter Highlights

This project demonstrates the ability to design a database schema, validate data quality using only SQL, and answer 17+ distinct business questions using a full range of SQL techniques — from simple aggregation to CTEs and window functions. It shows the candidate can work directly against a relational database without relying on Python or a BI tool, and can translate raw query output into specific, business-relevant recommendations — a core requirement for SQL-heavy Data Analyst roles.

---

## 32. Appendix

### SQL Queries

The complete SQL script is provided in `TASK-3_SQL_Queries.sql`, including database creation, table creation, all data cleaning checks, and all 17 business question queries documented in Section 13.

### Data Dictionary

See Section 6 ("Dataset Description") for the complete column-by-column data dictionary.

### Glossary

- **CTE (Common Table Expression):** A named temporary result set defined using a `WITH` clause, used to simplify complex queries.
- **Window Function:** A SQL function that performs a calculation across a set of rows related to the current row, without collapsing them into a single output row (e.g., `RANK()`, `LAG()`).
- **Primary Key:** A column (or set of columns) that uniquely identifies each row in a table.

### References

- Dataset: `Task-3_Dataset.csv` (project-provided file)
- SQL Script: `TASK-3_SQL_Queries.sql` (project-provided file)